



CYBERCLASS

Capacitando el presente para el futuro

Participante: Darien Ruben Suazo De Jesus

Técnico: Ciberseguridad

Módulo: CRIPTOGRAFÍA Y CIFRADO DE DATOS

Facilitador: Kevin Feliz Henríquez

Centro: Cyberclass

Práctica: Fundamentos de Criptografía

Descripción: En esta documentación verán el paso a paso de como hacer un programa que cifre y descifre el cifrado cesar

≡ Menú de Selección e Interfaz

app.py > mostrar_menu()

```
def mostrar_menu():
    print("\n===== ROT13 =====")
    print(f"      (Consultas realizadas: {len(historial)})")
    print("=====")
    print("1. Transformar mensaje")
    print("2. Destransformar mensaje")
    print("3. Historial")
    # ... otras opciones
```

Control de Estado en Vivo

La función modularizada de presentación de menú renderiza la interfaz en terminal utilizando:

- **Contador Dinámico:** Evalúa la longitud de la lista de memoria historial en cada iteración para reflejar las consultas ejecutadas.
- **Estructura Limpia:** Diseño ASCII alineado para proporcionar una experiencia de usuario (UX) intuitiva en CLI.

⚙️ Lógica de Rotación Híbrida ROT13/5

app.py > rot13()

```
def rot13(texto):
    resultado = ""
    for char in texto:
        if 'a' <= char <= 'z':
            resultado += chr((ord(char) - ord('a') + 13) % 26 + ord('a'))
        elif 'A' <= char <= 'Z':
            resultado += chr((ord(char) - ord('A') + 13) % 26 + ord('A'))
        elif '0' <= char <= '9':
            resultado += chr((ord(char) - ord('0') + 5) % 10 + ord('0'))
    return resultado
```

Procesamiento por Rangos

El código implementa un diseño de sustitución robusto:

- **Letras (ROT13):** Desplaza caracteres alfabéticos por 13 posiciones dentro de un módulo 26.
- **Dígitos Numéricos (ROT5):** Aplica rotación de 5 posiciones sobre un módulo 10 para cifrar números de forma segura.
- **Caracteres Especiales:** Se mantienen inalterados sin generar rupturas sintácticas.

↻ Transformación de Mensaje Activo

Caso de Prueba: Opción 1

Flujo de ejecución secuencial en tiempo de ejecución:

- **Mensaje Original:** hola como estas?
- **Rotación Aplicada:** Desplazamiento simétrico de +13 por carácter.
- **Mensaje Cifrado:** ubyn pbzb rfgnf?

Nótese que los espacios y el signo de interrogación final se preservan intactos tal como especifica el algoritmo.

Terminal - Opción 1

```
Selecione una opción: 1
Ingrese el mensaje a transformar: hola como estas?
Mensaje transformado: ubyn pbzb rfgnf?
```

🔄 Simetría Involutiva de Descifrado

```
app.py > desrot13()
```

```
def desrot13(texto):  
    return rot13(texto)
```

La Belleza de la Simetría

Una función involutiva es aquella que es su propia inversa:

Dado que:

$$f(f(x)) = x$$

No se necesita programar un algoritmo de descifrado separado.

Ejecutar la función sobre el texto cifrado `ubyn pbzb rfgnf?`

devuelve inmediatamente `hola como estas?`.

☰ Monitorización del Historial

```
● ● ● app.py > mostrar_historial()
```

```
def mostrar_historial(historial):  
    if not historial:  
        print("\nNo hay mensajes en el historial.")  
    else:  
        print("\n==== HISTORIAL =====")  
        for i, mensaje in enumerate(historial, 1):  
            print(f"{i}. {mensaje}")
```

Iteración Controlada

La opción 3 expone el log estructurado de la aplicación:

- **Evasión de Excepciones:** Verifica primero si el arreglo enlazado tiene elementos para evitar renderizados vacíos.
- **Formateador Dinámico:** Utiliza `enumerate(historial, 1)` para enumerar secuencialmente a partir de 1 sin sobrecargar memoria.

Persistencia de Datos en Disco

```
● ● ● app.py > guardar_historial()
```

```
def guardar_historial():  
    with open("historial.txt", "w") as archivo:  
        for mensaje in historial:  
            archivo.write(mensaje + "\n")
```

Gestor Contextual de Archivos

La función de guardado garantiza un volcado seguro a disco físico:

- **Context Manager (`with`)**: Garantiza el cierre automático del descriptor de archivo, incluso ante excepciones en el loop.
- **Escritura Lineal**: Serializa cada elemento añadiendo el delimitador de nueva línea estándar `\n`.

Verificación del Sistema de Archivos

 rot13

 app.py (Código fuente)

 **historial.txt (Salida)**

Estructura del Proyecto

Al invocar la opción 4 en la terminal, se genera instantáneamente el archivo plano historial.txt en el mismo directorio de ejecución (Path local).

Esto permite a la suite portar los registros sin depender de pesados motores de bases de datos relacionales externos.

Liberación de Recursos en Memoria

```
app.py > limpiar_historial()
```

```
def limpiar_historial():  
    historial.clear()  
    print("\nHistorial limpiado.")
```

Vaciado en Tiempo Real

El comando 5 ejecuta una limpieza de la estructura de datos dinámica:

- **Método Nativo:** `clear()` libera de forma eficiente las referencias del Garbage Collector de Python.
- **Reinicio de Estados:** Al volver al menú principal, el conteo de consultas realizadas se restablece instantáneamente a cero.

Resumen de lo aprendido

- ▶ Como funciona el cifrado rot13
- ▶ Como guardar los resultados de las consultas
- ▶ Implementacion del rot13 en un programa de cifrado